AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING • i-CREDIT HOURS PROGRAMS
Computer Engineering & Software Systems / AI Engineering

Assignment 1: MicroGrad Scalar Autograd Engine

CSE485 / CSE488: Deep Learning — Technical & Mathematical Report

Student Name: Madonna Emad Labib	**Student ID:** 23P0325	**GitHub:** @madonnaemad16
Term: Fall 2026	**Branch:** madonna-emad-labib_23P0325	**Autograder Score:** 12/12 PASSED (100%)

1. Executive Summary & Architecture

This report provides mathematical derivations, computational graph mechanics, and design justifications for building a lightweight scalar automatic differentiation (autograd) engine from first principles, inspired by Andrej Karpathy's micrograd. The engine enables reverse-mode automatic differentiation (backpropagation) over dynamic Directed Acyclic Graphs (DAGs) without third-party deep learning libraries. Overloaded dunder operators dynamically record node dependencies, enabling seamless gradient backpropagation via topological ordering.

2. Mathematical Derivations of Local Derivatives

In reverse-mode automatic differentiation, every node *v* receives an incoming adjoint derivative ∂L/∂v from its parent nodes, where *L* denotes the scalar objective (loss). The node applies the multivariate chain rule to distribute gradients to its direct antecedent children:

$$\partial L/\partial x_i = \sum_{\text{parents } p} (\partial L/\partial p) \times (\partial p/\partial x_i)$$

Operation	Forward Equation	Local Derivative Formula (∂z/∂x)	Python Closure Code
Addition (+)	z = x + y	∂z/∂x = 1.0 ∂z/∂y = 1.0	self.grad += 1.0 * out.grad other.grad += 1.0 * out.grad
Multiplication (*)	z = x × y	∂z/∂x = y ∂z/∂y = x	self.grad += other.data * out.grad other.grad += self.data * out.grad
Power (**)	$z = x^c$ ($c \in \Re$)	$\partial z/\partial x = c \times x^{c-1}$	self.grad += (c * (self.data**(c-1))) * out.grad
Hyperbolic Tangent	$z = \tanh(x) = (e^{2x}-1)/(e^{2x}+1)$	$\partial z/\partial x = 1 - \tanh^2(x) = 1 - z^2$	self.grad += (1.0 - z.data**2) * out.grad
Exponential	$z = \exp(x) = e^x$	$\partial z/\partial x = e^x = z$	self.grad += out.data * out.grad
Division (/)	$z = x / y = x \times y^{-1}$	Composed via Multiplication & Power	Implemented as: self * (other ** -1.0)

3. Graph Traversal & Mastering Topological Sort

To compute gradients in reverse mode, a node can calculate its total accumulated gradient only *after* all successor nodes depending on it have finished propagating their adjoints. This dependency order is strictly resolved using a **Depth-First Search (DFS) Topological Sort**.

Algorithm Formulation:

1. Maintain a list topo and a hash set visited.
2. For a given root node, visit child nodes in node._prev recursively.
3. Append the node to topo **only after** all recursive calls on child dependencies return (Post-Order DFS).
4. Reverse topo so that the graph evaluates from the final loss root back to the leaf inputs.

Critical TA Warnings & Edge Cases:

• **The Diamond Graph Trap (Gradient Accumulation):** If a variable is reused in multiple branches (e.g., c = a + a or d = c * a), the total derivative is the sum of partial paths: $\partial L/\partial a = \sum (\partial L/\partial path_i)$. Using assignment (=) overwrites earlier pathways. Using accumulation (self.grad += ...) correctly accumulates multivariable chain rule contributions.
• **The Post-Order Trap:** Appending nodes before visiting children reverses the evaluation dependency, causing parent nodes to execute backprop before child adjoints are available, resulting in zeroed or truncated gradients.
• **The Cycle / Re-visitation Trap:** Maintaining a global visited set guarantees that shared variables are traversed only once during the topological ordering, preventing infinite recursion.

4. Neural Network API Architecture

The src/nn.py module mirrors PyTorch's modular object-oriented design:

- **Neuron:** Initializes n_{in} weights and a bias drawn uniformly from *[-1.0, 1.0]*. Evaluates $tanh(\sum w_i x_i + b)$.
  - **Layer:** Encapsulates n_{out} parallel Neurons operating over the same feature input vector.
  - **MLP (Multi-Layer Perceptron):** Chains multiple Layers sequentially, passing intermediate activation outputs.
  - **zero_grad():** Iterates through all constituent Value instances and clears gradients to 0.0 prior to each backprop pass.

5. Experimental Results & Verification

The engine was comprehensively tested across unit tests and an end-to-end non-linear classification task:

Component / Test	Verification Criterion	Observed Result	Status
Pytest Suite (test_engine.py)	Arithmetic, type errors, diamond graph, power, tanh, exp	9/9 tests passed in 0.05s	**PASSED**
Pytest Suite (test_nn.py)	Neuron, Layer, and MLP parameter counts & forward/backward	3/3 tests passed in 0.02s	**PASSED**
GitHub CI/CD Autograder	Instructor PR pytest pipeline + Google Sheets sync	Score: 12/12 PASSED (Commit 1d3a82e)	**VERIFIED**
Non-Linear Classification	Binary concentric circles, MLP(2, [8, 8, 1]), Hinge Loss + L2	Loss: 0.88 → 0.02, Accuracy: 100%	**CONVERGED**

6. Conclusion

By implementing MicroGrad from scratch, we demonstrated how scalar automatic differentiation seamlessly executes the multivariate chain rule across arbitrary computational topologies. The engine handles gradient accumulation correctly, prevents circular traversals via topological sorting, and serves as an intuitive foundation for understanding modern deep learning engines like PyTorch.